

MongoDB Update Operators

Each question is shown in the blue box, followed by its answer. Commands are for mongosh. Each answer includes a small sample document so you can test the command yourself. ObjectIds and dates in the outputs are examples.

Question 1

Question 1. In a MongoDB collection named *playlists* (think Spotify), use the `$set` operator to update the 'genre' field to 'Chill' for the playlist with name 'Evening Vibes'.

Sample data (the same playlist is used again in Question 3):

```
use spotify_clone

db.playlists.insertOne({
  _id: 101,
  name: "Evening Vibes",
  genre: "Lo-Fi",
  songs: ["Ocean Eyes", "Sunset Lover"]
})
```

Answer: `$set` replaces the value of a field (or adds the field if it does not exist) without touching the other fields.

```
db.playlists.updateOne(
  { name: "Evening Vibes" },
  { $set: { genre: "Chill" } }
)
```

Output:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Verify:

```
db.playlists.findOne({ name: "Evening Vibes" })

{
  _id: 101,
  name: 'Evening Vibes',
  genre: 'Chill',
  songs: [ 'Ocean Eyes', 'Sunset Lover' ]
}
```

Question 2

Question 2. For a collection called *restaurants* (Zomato-style), remove the 'temporaryOffer' field from all documents where the offer has expired using the `$unset` operator.

Assumption: each temporary offer stores its end date in an `expiresOn` field inside `temporaryOffer`, so "expired" means `expiresOn` is earlier than the current date.

Sample data (assuming today is 24 September 2026):

```
use foodieApp

db.restaurants.insertMany([
  { name: "Burger Hub", cuisine: "Fast Food",
    temporaryOffer: { title: "20% off", expiresOn: ISODate("2026-08-31") } },
  ...
])
```

```

    { name: "Spice Villa", cuisine: "North Indian",
      temporaryOffer: { title: "Free dessert", expiresOn: ISODate("2026-12-31") } },
    { name: "Pizza Corner", cuisine: "Italian",
      temporaryOffer: { title: "Buy 1 Get 1", expiresOn: ISODate("2026-09-01") } },
    { name: "South Spice", cuisine: "South Indian" }
  ]
})

```

Answer: use `updateMany()` because many documents may match. The filter finds expired offers with `$lt` and `new Date()`, and `$unset` removes the field.

```

db.restaurants.updateMany(
  { "temporaryOffer.expiresOn": { $lt: new Date() } },
  { $unset: { temporaryOffer: "" } }
)

```

Output:

```

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}

```

Two documents (Burger Hub and Pizza Corner) had expired offers. Verify:

```

db.restaurants.find({}, { _id: 0 })

[
  { name: 'Burger Hub', cuisine: 'Fast Food' },
  {
    name: 'Spice Villa',
    cuisine: 'North Indian',
    temporaryOffer: { title: 'Free dessert', expiresOn: ISODate('2026-12-31T00:00:00.000Z') }
  },
  { name: 'Pizza Corner', cuisine: 'Italian' },
  { name: 'South Spice', cuisine: 'South Indian' }
]

```

Points to remember:

- The value given to `$unset` (here `""`) is ignored. Only the field name matters.
- Spice Villa keeps its offer because it has not expired yet, and South Spice has no offer, so the filter does not match it.
- The result depends on the date you run the command, because `new Date()` is the current date and time.
- If your data instead has a flag such as `offerExpired: true`, change the filter to `{ offerExpired: true }`.

Question 3

Question 3. Using the `$push` operator, add a new song 'Calm Waters' to the 'songs' array of the playlist document with `_id: 101` in your playlists collection. Hint: Use `$push` to add the song to the existing array without replacing it.

Answer: `$push` appends a value to the end of an existing array. The filter uses the number 101 (not a string), because that is how the `_id` was stored.

```

db.playlists.updateOne(
  { _id: 101 },
  { $push: { songs: "Calm Waters" } }
)

```

Output:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Verify:

```
db.playlists.findOne({ _id: 101 })
```

```
{
  _id: 101,
  name: 'Evening Vibes',
  genre: 'Chill',
  songs: [ 'Ocean Eyes', 'Sunset Lover', 'Calm Waters' ]
}
```

The existing songs are kept and "Calm Waters" is added at the end. If you used `$set: { songs: "Calm Waters" }` instead, it would replace the whole array. Note that `$push` adds the value every time, even if it is already in the array, which is the difference from `$addToSet` in the next question.

Question 4

Question 4. In a Flipkart-like products collection, use the `$addToSet` operator to ensure the tag 'trending' is added only once to the 'tags' array for the product with SKU 'F12345', even if you run the update multiple times.

Sample data (this product is also used in Question 5):

```
use flipkart_db
```

```
db.products.insertOne({
  sku: "F12345",
  name: "Wireless Earbuds",
  price: 1499,
  stock: 3,
  tags: ["audio", "bluetooth"]
})
```

Answer: `$addToSet` adds a value to an array only if it is not already there.

```
db.products.updateOne(
  { sku: "F12345" },
  { $addToSet: { tags: "trending" } }
)
```

Output of the first run (the tag is added):

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Output of running the same command again (nothing changes, because "trending" already exists):

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Verify:

```
db.products.findOne({ sku: "F12345" }, { _id: 0, sku: 1, tags: 1 })
```

```
{ sku: 'F12345', tags: [ 'audio', 'bluetooth', 'trending' ] }
```

"trending" appears only once. With \$push, each run would add another copy.

Question 5

Question 5. Simulate a flash sale: in your products collection, use the \$inc operator to decrease the 'stock' field by 1 for the product with name 'Wireless Earbuds' every time a purchase is made. Constraint: Do not allow stock to go below zero.

Answer: \$inc with -1 decreases the number by 1. To keep stock from going below zero, add stock: { \$gt: 0 } to the filter, so the update only runs while at least 1 item is left.

```
db.products.updateOne(
  { name: "Wireless Earbuds", stock: { $gt: 0 } },
  { $inc: { stock: -1 } }
)
```

What happens with stock = 3 (from the sample data):

Purchase	Stock before	matchedCount	modifiedCount	Stock after
1st	3	1	1	2
2nd	2	1	1	1
3rd	1	1	1	0
4th	0	0	0	0 (unchanged)

Output of the 4th purchase (no document matched, so the sale is refused and stock stays at 0):

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

Checking the result in code so the app can tell the customer what happened:

```
const result = db.products.updateOne(
  { name: "Wireless Earbuds", stock: { $gt: 0 } },
  { $inc: { stock: -1 } }
)

if (result.modifiedCount === 1) {
  print("Purchase successful");
} else {
  print("Sorry, Wireless Earbuds are out of stock");
}
```

Why this is the right approach:

- The check ("is stock above 0?") and the change ("subtract 1") happen together in one operation on one document, and MongoDB applies single-document updates atomically. Two customers buying the last item at the same moment cannot both succeed.
- Reading the stock first and then updating it in a second command would be unsafe, because the value could change in between during a busy flash sale.

Operator Summary

Operator	What it does
\$set	Sets a field to a value (adds it if missing)
\$unset	Removes a field from the document
\$push	Appends a value to an array (duplicates allowed)
\$addToSet	Adds a value to an array only if it is not already present
\$inc	Increases (or with a negative number, decreases) a numeric field